

PROGRAMACIÓN III

Guía Completa de Teoría

HTML · CSS · JavaScript · Flexbox · Fetch · Clases · Web Components

Parcial: 9 de Junio de 2025

UTN · Tecnicatura Universitaria en Programación

PARTE 1: HTML — Estructura y Semántica

HTML (HyperText Markup Language) es el lenguaje que define la estructura de una página web. Cada elemento HTML es una etiqueta que el navegador interpreta como un objeto del DOM (Document Object Model).

1.1 Estructura base de un documento HTML

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Título de la página</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <!-- Contenido visible de la página -->
    <script src="main.js"></script> <!-- al final del body -->
  </body>
</html>
```

Etiqueta	Para qué sirve
<!DOCTYPE html>	Le dice al navegador que es un documento HTML5. Va siempre primero.
<html lang="es">	Raíz del documento. El atributo lang ayuda a lectores de pantalla y buscadores.
<head>	Metadatos: charset, viewport, título, links a CSS. No se ve en la página.
<meta charset="UTF-8">	Permite usar tildes y caracteres especiales correctamente.
<meta name="viewport">	Fundamental para que el diseño funcione bien en celulares.
<body>	Todo lo que se ve en pantalla va acá.
<script> al final del body	Espera a que el HTML esté cargado antes de ejecutar JS. Evita errores de DOM.

1.2 Etiquetas semánticas

Las etiquetas semánticas describen el significado del contenido, no solo su apariencia. Ayudan a los buscadores, los lectores de pantalla y a otros desarrolladores a entender la estructura de la página.

Etiqueta	Significado	Uso típico
<header>	Cabecera de la página o sección	Logo, título, navegación principal
<nav>	Bloque de navegación	Menú con links de la página
<main>	Contenido principal (único por página)	El bloque central de contenido
<section>	Sección temática del contenido	Agrupación de contenido relacionado
<article>	Contenido independiente y reutilizable	Entradas de blog, tarjetas, noticias
<aside>	Contenido secundario o complementario	Sidebar, publicidades, links relacionados
<footer>	Pie de página o sección	Datos de contacto, copyright
<h1>...<h6>	Títulos de distinta jerarquía	Solo un h1 por página; los demás estructuran
<p>	Párrafo de texto	Texto corrido
<div>	Contenedor genérico sin semántica	Para agrupar y dar estilos (no tiene significado)
	Contenedor inline sin semántica	Para aplicar estilos a parte de un texto

1.3 El DOM — Document Object Model

El DOM es la representación que el navegador hace del HTML como un árbol de objetos JavaScript. Cada etiqueta HTML se convierte en un nodo del DOM que JavaScript puede leer y modificar.

Concepto clave: "etiqueta = instancia". Cuando el navegador lee <button id='btn'>, crea un objeto JavaScript de tipo HTMLButtonElement. Podés acceder a él con document.getElementById('btn').

1.4 Formularios y elementos de entrada

```

<form>
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" placeholder="Tu nombre">

  <select id="dificultad">
    <option value="easy">Fácil</option>
    <option value="medium">Medio</option>
    <option value="hard">Difícil</option>
  </select>

```

```
<button type="button" id="btn-jugar">Jugar</button>  
</form>
```

⚠ Importante: Usá type="button" en botones dentro de formularios para evitar que recarguen la página. Sin ese atributo, el botón hace submit por defecto.

PARTE 2: CSS — Estilos y Box Model

CSS (Cascading Style Sheets) controla la apariencia visual de los elementos HTML. La palabra 'cascading' (en cascada) significa que los estilos se aplican en orden de especificidad y posición.

2.1 Selectores

Selector	Ejemplo	Selecciona
Etiqueta	<code>p { }</code>	Todos los párrafos
Clase (.)	<code>.card { }</code>	Todos los elementos con <code>class="card"</code>
ID (#)	<code>#hero { }</code>	El único elemento con <code>id="hero"</code>
Descendiente	<code>.nav a { }</code>	Links dentro de <code>.nav</code>
Hijo directo (>)	<code>.menu > li { }</code>	Li que son hijos directos de <code>.menu</code>
Pseudoclase (:hover)	<code>button:hover { }</code>	El botón cuando el mouse está encima
Pseudoclase (:first-child)	<code>li:first-child { }</code>	El primer li de su padre
Universal (*)	<code>* { box-sizing: border-box; }</code>	Todos los elementos

2.2 El Box Model

Todo elemento HTML es una caja rectangular con cuatro capas. De adentro hacia afuera: content → padding → border → margin.

```
.caja {
  width: 200px;           /* ancho del contenido */
  padding: 16px;          /* espacio INTERNO (empuja el borde hacia
afuera) */
  border: 2px solid #333;
  margin: 24px;           /* espacio EXTERNO (aleja el elemento de sus
vecinos) */
  box-sizing: border-box; /* el ancho incluye padding y border —
SIEMPRE usarlo */
}
```

Propiedad	Qué controla
width / height	Dimensiones del área de contenido.
padding	Espacio entre el contenido y el borde. Tiene el color de fondo del elemento.

Propiedad	Qué controla
border	El borde visible del elemento. Suma al tamaño total (salvo con box-sizing: border-box).
margin	Espacio externo entre este elemento y sus vecinos. Es transparente.
box-sizing: border-box	El width definido incluye padding + border. Hace los cálculos predecibles.

2.3 Display — cómo se comporta cada elemento

Valor	Comportamiento	Ejemplos de elementos
block	Ocupa toda la línea. Apila verticalmente. Acepta width/height.	div, p, h1-h6, section, header
inline	Solo ocupa lo que necesita. No acepta width/height. No genera salto de línea.	span, a, strong, em
inline-block	Como inline (en línea) pero acepta width/height.	img, button (por defecto)
flex	Activa Flexbox en el contenedor. Sus hijos son flex items.	Contenedores de layout
grid	Activa CSS Grid. Layout en dos dimensiones.	Contenedores de cuadrícula
none	El elemento desaparece completamente del DOM visible.	Ocultar elementos

2.4 Position

Valor	Comportamiento
static (default)	Flujo normal. top/left/right/bottom no tienen efecto.
relative	Se desplaza DESDE su posición normal. El espacio que ocupaba se mantiene.
absolute	Se posiciona respecto al ancestro con position relativo más cercano. Sale del flujo.
fixed	Se posiciona respecto a la ventana del navegador. No se mueve al hacer scroll.
sticky	Relative hasta que llega a un umbral del scroll, luego se comporta como fixed.

2.5 Variables CSS (Custom Properties)

Las variables CSS permiten definir valores reutilizables. Se declaran con -- y se usan con var().

```
:root {
  --color-primario: #1F4E79;
```

```
--color-acento:    #00d4ff;
--fuente-mono:     'Space Mono', monospace;
--espaciado-base: 16px;
}

.boton {
  background: var(--color-primario);
  color: var(--color-acento);
  font-family: var(--fuente-mono);
  padding: var(--espaciado-base);
}
```

□ **Tip:** Siempre declarar las variables globales en :root. Así quedan disponibles en todo el documento.

PARTE 3: Flexbox — Layout Unidimensional

Flexbox (Flexible Box Layout) es un modelo de layout de CSS que permite distribuir elementos en una fila o columna, controlando alineación y espacio de forma flexible. Se activa en el CONTENEDOR con `display: flex`. Sus hijos pasan a ser flex items automáticamente.

3.1 Propiedades del CONTENEDOR flex

Propiedad	Valores principales	Qué hace
<code>display: flex</code>	—	Activa Flexbox. Los hijos directos pasan a ser flex items.
<code>flex-direction</code>	<code>row</code> <code>column</code> <code>row-reverse</code> <code>column-reverse</code>	Define si los items se acomodan en fila (por defecto) o columna.
<code>justify-content</code>	<code>flex-start</code> <code>flex-end</code> <code>center</code> <code>space-between</code> <code>space-around</code> <code>space-evenly</code>	Alinea los items sobre el eje PRINCIPAL (horizontal en row).
<code>align-items</code>	<code>flex-start</code> <code>flex-end</code> <code>center</code> <code>stretch</code> <code>baseline</code>	Alinea los items sobre el eje CRUZADO (vertical en row).
<code>flex-wrap</code>	<code>nowrap</code> <code>wrap</code> <code>wrap-reverse</code>	Si los items se acomodan en una o varias líneas cuando no caben.
<code>gap</code>	Ej: 16px 16px 24px	Espacio entre items. Reemplaza a los márgenes entre items.
<code>align-content</code>	<code>flex-start</code> <code>center</code> <code>space-between</code> etc.	Alinea las FILAS cuando hay flex-wrap y hay espacio extra.

3.2 Propiedades del ITEM flex

Propiedad	Valores	Qué hace
<code>flex-grow</code>	Número (ej: 1)	Cuánto crece el item respecto a los demás para llenar el espacio libre.
<code>flex-shrink</code>	Número (ej: 0)	Cuánto se encoge si el espacio es insuficiente. 0 = no se encoge.
<code>flex-basis</code>	Ej: 200px auto	Tamaño base del item antes de distribuir espacio libre.
<code>flex</code>	Shorthand: <code>flex-grow flex-shrink flex-basis</code>	Ej: <code>flex: 1 1 0%</code> (crece y se encoge equitativamente).
<code>align-self</code>	<code>auto</code> <code>flex-start</code> <code>center</code> <code>flex-end</code> <code>stretch</code>	Sobreescribe <code>align-items</code> solo para este item.
<code>order</code>	Número entero	Cambia el orden visual sin cambiar el HTML. Por defecto 0.

3.3 Ejemplos fundamentales

Barra de navegación

```
.navbar {
  display: flex;
  justify-content: space-between; /* logo izq, links centro/der */
  align-items: center;           /* centra verticalmente */
  padding: 0 24px;
  height: 60px;
}
```

Grilla de tarjetas que pasan a nueva línea

```
.cards-container {
  display: flex;
  flex-wrap: wrap; /* permite múltiples filas */
  gap: 16px;
}
.card {
  flex: 1 1 200px; /* crece, se encoge, base 200px */
  /* cada card ocupa al menos 200px y crece para llenar el espacio */
}
```

Sidebar fijo + contenido flexible

```
.layout {
  display: flex;
  min-height: 100vh;
}
.sidebar {
  width: 260px;
  flex-shrink: 0; /* no se encoge */
}
.main-content {
  flex: 1; /* ocupa todo el espacio restante */
}
```

Layout de página completa (header + main + footer)

```
body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
main {
  flex: 1; /* el main crece para empujar el footer al fondo */
}
```

}

3.4 Cheat sheet: justify-content

Valor	Comportamiento visual
flex-start	Items pegados al inicio (izquierda en row). Es el default.
flex-end	Items pegados al final (derecha en row).
center	Items centrados en el eje principal.
space-between	Primer item al inicio, último al final. Espacio igual ENTRE items.
space-around	Espacio igual alrededor de cada item (los extremos tienen la mitad).
space-evenly	Espacio exactamente igual entre todos, incluyendo los extremos.

⚠ Importante: flex-direction: column cambia el eje principal a vertical. Entonces justify-content controla la alineación VERTICAL y align-items controla la HORIZONTAL. ¡Al revés de lo que parece intuitivo!

PARTE 4: Responsividad — Media Queries y Container Queries

Un diseño responsivo se adapta a distintos tamaños de pantalla. El enfoque moderno parte del diseño para celulares (mobile first) y va agregando estilos para pantallas más grandes.

4.1 Media Queries

Las media queries permiten aplicar estilos solo cuando se cumple una condición de pantalla.

```
/* Mobile first: estilos base para celular */
.cards { flex-direction: column; }

/* Tablet (768px o más) */
@media (min-width: 768px) {
  .cards { flex-direction: row; flex-wrap: wrap; }
}

/* Desktop (1200px o más) */
@media (min-width: 1200px) {
  .cards { gap: 32px; }
  .sidebar { display: block; }
}
```

Breakpoint	Ancho	Dispositivo típico
Base (sin query)	< 576px	Celulares chicos
sm	≥ 576px	Celulares grandes
md	≥ 768px	Tablets
lg	≥ 1024px	Laptops
xl	≥ 1200px	Desktops

4.2 Container Queries (moderno)

A diferencia de las media queries que miran el viewport, las container queries miran el tamaño del CONTENEDOR del elemento. Perfectas para componentes reutilizables.

```
/* 1. Definir el contenedor */
.wrapper {
  container-type: inline-size;
  container-name: card-container; /* opcional */
}

/* 2. Regla que se aplica cuando el contenedor mide al menos 400px */
@container (min-width: 400px) {
```

```
.card { flex-direction: row; }  
}
```

□ **Tip:** Las container queries son perfectas para el TP: una card puede cambiar su layout según el tamaño de su contenedor, sin importar el tamaño del viewport.

PARTE 5: JavaScript — Clases y POO

Las clases en JavaScript son plantillas para crear objetos. Agrupan datos (propiedades) y comportamiento (métodos) en una sola entidad reutilizable.

5.1 Anatomía de una clase

```
class Personaje {
  // Propiedad estática: pertenece a la CLASE, no a cada instancia
  static ESPECIE_DEFAULT = 'Humano';

  // Constructor: se ejecuta automáticamente al hacer new Personaje()
  constructor(nombre, especie) {
    this.nombre = nombre;    // propiedad pública de instancia
    this.especie = especie;
    this.#vida = 100;        // propiedad PRIVADA (solo accesible
                              dentro de la clase)
  }

  // Método público
  presentarse() {
    return `Soy ${this.nombre}, un ${this.especie}`;
  }

  // Método privado (ES2022)
  #calcularDaño() { return Math.random() * 10; }

  // Getter: acceso como propiedad pero ejecuta código
  get estaVivo() { return this.#vida > 0; }
}

// Instanciar:
const rick = new Personaje('Rick', 'Humano');
console.log(rick.presentarse());    // 'Soy Rick, un Humano'
console.log(rick.estaVivo);         // true
console.log(Personaje.ESPECIE_DEFAULT); // 'Humano' (acceso por clase,
no instancia)
```

5.2 Tabla de conceptos de clase

Concepto	Sintaxis	Qué es
constructor	constructor(param) { }	Se ejecuta al hacer new. Inicializa las propiedades.
this	this.nombre = valor	Referencia a la instancia actual del objeto.

Concepto	Sintaxis	Qué es
Método	metodo() {}	Función que vive en la clase y tiene acceso a this.
Propiedad estática	static BASE_URL = '...'	Pertenece a la clase, no a la instancia. Se accede con Clase.propiedad.
Método estático	static crearDefault() {}	Se llama en la clase directamente: Clase.crearDefault().
Propiedad privada	#nombre = valor	Solo accesible dentro de la clase. Error si se accede desde afuera.
Método privado	#calcular() {}	Solo puede llamarse desde dentro de la clase.
Getter	get propiedad() { return x; }	Se accede como propiedad (sin paréntesis) pero ejecuta código.

5.3 Clase con método async — Encapsular fetch

Los métodos de una clase también pueden ser async. Esta es la base del patrón que usa el TP (TriviaAPI y RickMortyAPI):

```
class TriviaAPI {
  static BASE_URL = 'https://opentdb.com';

  async getPreguntas(cantidad = 10, categoria = '', dificultad = '') {
    // Construir la URL con los parámetros opcionales:
    let url =
      `${TriviaAPI.BASE_URL}/api.php?amount=${cantidad}&type=multiple`;
    if (categoria) url += `&category=${categoria}`;
    if (dificultad) url += `&difficulty=${dificultad}`;

    try {
      const response = await fetch(url);
      if (!response.ok) throw new Error(`Error HTTP:
      ${response.status}`);
      const data = await response.json();
      return data.results; // array de preguntas
    } catch (error) {
      console.error('Error en getPreguntas:', error);
      throw error; // re-lanzar para que quien llame pueda manejarlo
    }
  }

  async getCategorias() {
    const response = await
    fetch(`${TriviaAPI.BASE_URL}/api_category.php`);
    if (!response.ok) throw new Error(`Error HTTP:
    ${response.status}`);
    const data = await response.json();
    return data.trivia_categories;
  }
}
```

```
}  
}
```

5.4 Clase de lógica — TriviaGame

Esta clase no hace fetch. Solo maneja el estado del juego: preguntas, índice actual, puntaje:

```
class TriviaGame {  
  constructor() {  
    this.preguntas      = [];  
    this.preguntaActual = 0;  
    this.puntaje        = 0;  
  }  
  
  iniciar(preguntas) {  
    this.preguntas      = preguntas;  
    this.preguntaActual = 0;  
    this.puntaje        = 0;  
  }  
  
  getPreguntaActual() {  
    return this.preguntas[this.preguntaActual];  
  }  
  
  responder(respuesta) {  
    const correcta = this.getPreguntaActual().correct_answer;  
    const esCorrecta = respuesta === correcta;  
    if (esCorrecta) this.puntaje++;  
    return esCorrecta; // devuelve true o false  
  }  
  
  siguiente() {  
    this.preguntaActual++;  
  }  
  
  haTerminado() {  
    return this.preguntaActual >= this.preguntas.length;  
  }  
}
```

PARTE 6: Asincronía — Promises, async/await y fetch()

JavaScript es single-threaded: ejecuta una sola cosa a la vez. Para operaciones lentas (como pedir datos a un servidor), usa asincronía: no bloquea el programa mientras espera la respuesta.

6.1 Promise — una promesa de resultado futuro

Una Promise representa un valor que todavía no existe pero existirá en el futuro. Tiene tres estados: pending (esperando), fulfilled (resuelta con éxito) o rejected (falló).

```
// Forma 1: .then() — cadena de promesas
fetch('https://rickandmortyapi.com/api/character/1')
  .then(response => response.json()) // parsea JSON
  .then(data => console.log(data))    // usa los datos
  .catch(error => console.error(error));

// Forma 2: async/await — la moderna (más legible)
async function obtenerPersonaje(id) {
  const response = await
  fetch(`https://rickandmortyapi.com/api/character/${id}`);
  const data = await response.json();
  return data;
}
// await 'pausa' la función hasta que la Promise se resuelve
// SOLO se puede usar await dentro de una función async
```

6.2 La anatomía completa de fetch()

```
async function fetchSeguro(url) {
  try {
    // 1. Hacer la petición:
    const response = await fetch(url);

    // 2. Verificar que el servidor respondió bien:
    //    fetch() NO lanza error en 404 o 500 — hay que checar
    response.ok
    if (!response.ok) {
      throw new Error(`Error HTTP ${response.status}:
      ${response.statusText}`);
    }

    // 3. Parsear el JSON:
    const data = await response.json();

    // 4. Devolver los datos:
    return data;
  }
}
```



```

    } catch (error) {
      // Atrapa: red caída, URL inválida, error 404/500 que lanzamos,
      etc.
      console.error('Falló la petición:', error.message);
      throw error; // re-lanzar para que el llamador pueda reaccionar
    }
  }
}

```

⚠ Importante: `fetch()` NUNCA lanza error aunque el servidor devuelva 404 o 500. Solo lanza error si hay problema de red o la URL es inválida. SIEMPRE verificar `response.ok` manualmente.

6.3 Manejo de errores en la UI

El TP pide mostrar estados de carga y error al usuario. El patrón típico:

```

async function cargarPreguntas() {
  // Mostrar estado de carga:
  document.getElementById('status').textContent = 'Cargando
  preguntas...';

  try {
    const preguntas = await api.getPreguntas(10);
    game.iniciar(preguntas);
    mostrarPregunta();
  } catch (error) {
    // Mostrar error legible (sin el stack técnico):
    document.getElementById('status').textContent =
      'No pudimos cargar las preguntas. Revisá tu conexión.';
    document.getElementById('btn-reintentar').style.display = 'block';
  }
}

```

6.4 Fisher-Yates Shuffle — mezclar array

El TP pide que las opciones de respuesta aparezcan en orden aleatorio. El algoritmo estándar para mezclar arrays es Fisher-Yates:

```

function mezclar(array) {
  // Copia el array para no modificar el original:
  const copia = [...array];

  // Recorre de atrás para adelante:
  for (let i = copia.length - 1; i > 0; i--) {
    // Elige un índice aleatorio entre 0 e i (inclusive):
    const j = Math.floor(Math.random() * (i + 1));

    // Intercambia los elementos en posición i y j:

```

```

        [copia[i], copia[j]] = [copia[j], copia[i]];
    }

    return copia;
}

// Uso para mezclar las 4 opciones:
const opciones = mezclar([
    pregunta.correct_answer,
    ...pregunta.incorrect_answers
]);

```

6.5 Decodificar HTML escapado

La API de Trivia devuelve textos con caracteres HTML escapados como &, ", '. Hay que decodificarlos antes de mostrarlos:

```

function decodificarHTML(texto) {
    // El textarea decodifica las entidades HTML automáticamente:
    const el = document.createElement('textarea');
    el.innerHTML = texto;
    return el.value;
}

// Uso:
const textoLimpio = decodificarHTML(pregunta.question);
document.getElementById('pregunta').textContent = textoLimpio;

```

PARTE 7: Web Components

Un Web Component es un elemento HTML personalizado que definís vos. Una vez creado, lo usás como cualquier etiqueta HTML nativa: `<mi-componente>`. Es nativo del navegador — no necesita frameworks.

7.1 Los tres pilares

Pilar	Qué es	Para qué sirve
Custom Elements	Registrar una nueva etiqueta HTML	Definir el comportamiento del componente
Shadow DOM	Un DOM aislado dentro del componente	Encapsular CSS para que no afecte ni sea afectado por el exterior
<code><template></code> y <code><slot></code>	HTML que no se renderiza hasta clonarlo	Definir el molde del componente; <code><slot></code> es un hueco para contenido externo

7.2 Ciclo de vida (Lifecycle Callbacks)

Callback	Cuándo se ejecuta
<code>constructor()</code>	Al crear la instancia (<code>new</code> o al parsear el HTML). Siempre llamar <code>super()</code> primero.
<code>connectedCallback()</code>	Cuando el elemento se AGREGA al DOM. Equivale al <code>DOMContentLoaded</code> pero para el componente. Acá se renderiza.
<code>disconnectedCallback()</code>	Cuando el elemento se ELIMINA del DOM. Útil para limpiar event listeners.
<code>attributeChangedCallback(name, oldVal, newVal)</code>	Cuando cambia un atributo observado. Solo los declarados en <code>observedAttributes()</code> se observan.
<code>static get observedAttributes()</code>	No es un callback, es un getter estático. Devuelve array de nombres de atributos a observar.

7.3 Custom Element mínimo

```
// 1. Crear clase que extiende HTMLElement:
class MiSaludo extends HTMLElement {
  connectedCallback() {
    this.innerHTML = '<h2>¡Hola desde un Web Component!</h2>';
  }
}

// 2. Registrar el elemento (nombre SIEMPRE con guión):
customElements.define('mi-saludo', MiSaludo);
```

```
// 3. Usar en HTML:  
// <mi-saludo></mi-saludo>
```

⚠ Importante: El nombre del custom element SIEMPRE debe tener al menos un guión (-). Es obligatorio: 'mi-saludo', 'character-card', 'tarjeta-info'. Sin guión, el navegador no lo reconoce como custom element.

7.4 Shadow DOM — aislamiento de estilos

```
class MiBoton extends HTMLElement {  
  constructor() {  
    super(); // siempre primero en el constructor  
  
    // Crear el shadow DOM (mode: 'open' permite acceder desde JS  
    externo):  
    const shadow = this.attachShadow({ mode: 'open' });  
  
    // El CSS acá NO afecta al resto de la página:  
    shadow.innerHTML = `  
      <style>  
        button {  
          background: #00d4ff;  
          border: none;  
          padding: 8px 16px;  
          border-radius: 4px;  
          font-weight: bold;  
          cursor: pointer;  
        }  
      </style>  
      <button>Click!</button>  
    `;  
  }  
}  
customElements.define('mi-boton', MiBoton);
```

7.5 Atributos reactivos — observedAttributes

```
class MiTarjeta extends HTMLElement {  
  
  // Declarar qué atributos observar:  
  static get observedAttributes() {  
    return ['nombre', 'especie', 'status'];  
  }  
  
  connectedCallback() {  
    this.render(); // renderizar cuando se agrega al DOM  
  }  
}
```

```

    }

    // Se llama automáticamente cuando cambia un atributo observado:
    attributeChangedCallback(nombre, valorAnterior, valorNuevo) {
        this.render(); // re-renderizar con los nuevos valores
    }

    render() {
        const nombre = this.getAttribute('nombre') || 'Sin nombre';
        const especie = this.getAttribute('especie') || 'Desconocido';
        const status = this.getAttribute('status') || 'unknown';

        this.innerHTML = `
            <div class="tarjeta">
                <h3>${nombre}</h3>
                <p>${especie} - ${status}</p>
            </div>
        `;
    }
}
customElements.define('mi-tarjeta', MiTarjeta);

// En HTML:
// <mi-tarjeta nombre="Rick" especie="Humano" status="Alive"></mi-
tarjeta>

```

7.6 <template> y <slot>

<template> define HTML que el navegador no renderiza inmediatamente. Se clona con JavaScript cuando se necesita. <slot> es un hueco donde se proyecta contenido del exterior del componente.

```

<!-- En el HTML: -->
<template id="tpl-tarjeta">
    <style>
        .tarjeta { border: 1px solid #ddd; border-radius: 10px; padding:
1rem; }
        h3 { color: #1D9E75; }
    </style>
    <div class="tarjeta">
        <h3><slot name="titulo">Título por defecto</slot></h3>
        <p><slot>Contenido por defecto</slot></p>
        <small><slot name="pie">Pie por defecto</slot></small>
    </div>
</template>

<!-- Uso en el HTML: -->
<articulo-card>
    <span slot="titulo">Introducción a Web Components</span>
    <p>Este es el contenido del artículo.</p>

```

```

    <span slot="pie">Publicado: 11 de Mayo, 2026</span>
  </articulo-card>

```

```

// En JavaScript:
class ArticuloCard extends HTMLElement {
  connectedCallback() {
    const shadow = this.attachShadow({ mode: 'open' });
    const template = document.getElementById('tpl-tarjeta');
    // cloneNode(true) = copia profunda (incluye todos los hijos):
    shadow.appendChild(template.content.cloneNode(true));
  }
}

customElements.define('articulo-card', ArticuloCard);

```

Concepto	Explicación
<code><template></code>	HTML que existe en el DOM pero NO se renderiza. Solo se activa al clonarlo.
<code>template.content</code>	El contenido del template como DocumentFragment.
<code>cloneNode(true)</code>	Copia profunda: incluye todos los hijos. <code>cloneNode(false)</code> solo copia el nodo raíz.
<code><slot></code>	Un hueco dentro del template. El contenido del usuario se proyecta ahí.
<code>slot="nombre"</code>	Named slot: el contenido con <code>slot="nombre"</code> va al <code><slot name="nombre"></code> .
Slot sin nombre	El contenido sin atributo slot va al primer <code><slot></code> sin nombre.

7.7 Ejemplo completo — CharacterCard (del proyecto de clase)

```

class CharacterCard extends HTMLElement {
  static get observedAttributes() {
    return ['name', 'image', 'status', 'species', 'origin', 'gender'];
  }

  constructor() {
    super();
    this.shadow = this.attachShadow({ mode: 'open' });
  }

  connectedCallback() { this.render(); }
  attributeChangedCallback() { this.render(); }

  render() {
    const g = (attr) => this.getAttribute(attr) || '-';
    const statusColor = { Alive: '#10b981', Dead: '#ef4444' };
    const status = g('status') || '#64748b';

    this.shadow.innerHTML = `
      <style>

```

```

        :host { display: block; }
        .card { background: #1a2236; border-radius: 10px; overflow:
hidden; }
        .card:hover { transform: translateY(-4px); }
        img { width: 100%; display: block; }
        .info { padding: 12px; }
        .name { font-weight: bold; color: #e2e8f0; }
        .dot { width: 7px; height: 7px; border-radius: 50%;
            background: ${statusColor}; display: inline-block; }
    </style>
    <div class="card">
        
        <div class="info">
            <div class="name">${g('name')}</div>
            <div><span class="dot"></span> ${g('status')} .
${g('species')}</div>
            <div>□ ${g('origin')}</div>
        </div>
    </div>
    `;
}
}
customElements.define('character-card', CharacterCard);

// Uso en HTML (generado dinámicamente con JS):
// <character-card name="Rick" image="url..." status="Alive"
species="Human" origin="Earth">
// </character-card>

```

PARTE 8: Manipulación del DOM con JavaScript

El DOM (Document Object Model) es el árbol de objetos que el navegador construye a partir del HTML. JavaScript puede leer y modificar cualquier elemento del DOM en tiempo real.

8.1 Seleccionar elementos

Método	Qué devuelve	Ejemplo
<code>getElementById('id')</code>	Un elemento (o null)	<code>document.getElementById('btn-jugar')</code>
<code>querySelector('selector')</code>	El PRIMER elemento que coincide (o null)	<code>document.querySelector('.card')</code>
<code>querySelectorAll('selector')</code>	NodeList de todos los que coinciden	<code>document.querySelectorAll('.opcion')</code>

8.2 Modificar el DOM

```
const elemento = document.getElementById('resultado');

// Cambiar el texto visible:
elemento.textContent = 'Nuevo texto'; // solo texto, no interpreta HTML

// Cambiar HTML interno (permite insertar tags):
elemento.innerHTML = '<strong>Texto en negrita</strong>';

// Cambiar estilos en línea:
elemento.style.color = 'red';
elemento.style.display = 'none';

// Agregar / quitar clases:
elemento.classList.add('activo');
elemento.classList.remove('oculto');
elemento.classList.toggle('seleccionado'); // agrega si no tiene, quita si tiene
elemento.classList.contains('activo');      // devuelve true o false

// Leer atributos:
const valor = elemento.getAttribute('data-id');
elemento.setAttribute('disabled', 'true');

// Crear y agregar elementos:
const nuevoDiv = document.createElement('div');
nuevoDiv.textContent = 'Soy nuevo';
document.body.appendChild(nuevoDiv);
```


8.3 Eventos

```
// Agregar event listener:
const boton = document.getElementById('btn-jugar');
boton.addEventListener('click', function() {
  console.log('¡Click!');
});

// Con función de flecha y acceso al evento:
boton.addEventListener('click', (event) => {
  event.preventDefault(); // evita el comportamiento por defecto
  console.log('Botón clickeado:', event.target);
});

// Evento 'input': se dispara en cada cambio del campo
document.getElementById('buscador').addEventListener('input', async (e)
=> {
  const texto = e.target.value;
  // llamar a la API con el texto...
});

// DOMContentLoaded: esperar a que el HTML esté listo
document.addEventListener('DOMContentLoaded', () => {
  // código que accede al DOM acá
});
```

8.4 innerHTML vs template literals — renderizar listas

La forma más común de mostrar datos de una API en el DOM es generar HTML con template literals y asignarlo a innerHTML:

```
async function mostrarPersonajes() {
  const personajes = await api.getCharacters();
  const contenedor = document.getElementById('grid');

  // map() transforma cada personaje en un string HTML:
  contenedor.innerHTML = personajes
    .map(p => `
      <div class="card">
        
        <h3>${p.name}</h3>
        <p>${p.species} · ${p.status}</p>
      </div>
    `).join(''); // join('') une el array de strings en uno solo
}
```

PARTE 9: Resumen Rápido — Para Repasar Antes del Parcial

HTML — Puntos clave

Pregunta	Respuesta
¿Qué es el DOM?	La representación del HTML como árbol de objetos JavaScript que el navegador crea.
¿Por qué <script> al final del body?	Para que el DOM esté disponible cuando el JS se ejecuta.
Diferencia div vs section	div no tiene semántica. section agrupa contenido temáticamente relacionado.
¿Para qué sirve type="button"?	Evita que el botón haga submit del formulario por defecto.

CSS y Flexbox — Puntos clave

Pregunta	Respuesta
¿Qué hace box-sizing: border-box?	El width incluye padding y border, haciendo los cálculos predecibles.
¿Qué hace display: flex?	Convierte el elemento en flex container. Sus hijos directos pasan a ser flex items.
justify-content vs align-items	justify-content = eje principal (horizontal en row). align-items = eje cruzado (vertical en row).
flex-wrap: wrap	Permite que los items pasen a una segunda fila cuando no caben en la primera.
flex-grow: 1	El item ocupa todo el espacio libre disponible. Si dos tienen flex-grow: 1, lo dividen.
flex-shrink: 0	El item no se encoge aunque el espacio sea limitado. Útil para sidebar fijo.
¿Qué es mobile first?	Escribir los estilos base para celular y agregar media queries para pantallas más grandes.
@container vs @media	@media mira el viewport. @container mira el tamaño del contenedor del elemento.

JavaScript y fetch() — Puntos clave

Pregunta	Respuesta
¿Por qué async antes de la función?	Para poder usar await dentro. Sin async, await da error de sintaxis.
¿Por qué verificar response.ok?	fetch() no lanza error en 404 o 500. Solo falla si hay problema de red.
¿Qué hace response.json()?	Parsea el body de la respuesta como JSON y devuelve un objeto JavaScript.
¿Qué hace throw error en catch?	Re-lanza el error para que el código que llamó a la función también pueda manejarlo.
¿Qué es una propiedad estática?	Pertenece a la CLASE, no a las instancias. Se accede con Clase.propiedad.
this en una clase	Referencia a la instancia actual del objeto.

Web Components — Puntos clave

Pregunta	Respuesta
¿Por qué el nombre lleva guión?	Es obligatorio para distinguir custom elements de etiquetas HTML nativas.
¿Qué hace connectedCallback()?	Se ejecuta cuando el elemento se agrega al DOM. Es donde se renderiza el componente.
¿Qué hace attributeChangedCallback()?	Se ejecuta cuando cambia un atributo observado. Permite re-renderizar al cambiar datos.
¿Qué hace observedAttributes?	Es un getter estático que devuelve qué atributos deben disparar attributeChangedCallback.
¿Qué es el Shadow DOM?	Un DOM aislado dentro del componente. El CSS no se escapa ni entra.
attachShadow({ mode: 'open' })	Crea el shadow DOM. mode: 'open' permite acceder al shadow desde JS externo.
¿Para qué sirve <template>?	Define HTML que no se renderiza. Se clona con template.content.cloneNode(true).
cloneNode(true) vs cloneNode(false)	true = copia profunda (incluye todos los hijos). false = solo el nodo raíz.
¿Qué es un <slot>?	Un hueco en el template donde se proyecta contenido externo del usuario del componente.
super() en constructor	Llama al constructor de HTMLElement. SIEMPRE primero, antes de cualquier otra cosa.